

Ejercicio 23: Reparto Óptimo de Galletas (Enfoque Voraz)

Enunciado

Tenemos que repartir galletas a unos niños sabiendo el tamaño mínimo de la galleta que quiere cada uno. Para ello, tenemos un vector D donde la posición i indica el tamaño mínimo de galleta que quiere el niño i . Además tenemos otro vector G , donde la posición j representa el tamaño de la galleta j . Queremos saber el máximo número de niños a los que podemos satisfacer con las galletas disponibles (le daremos a cada uno una galleta como máximo).

Un niño i estará satisfecho con la galleta j si su tamaño es igual o mayor al solicitado:

$$D[i] \leq G[j]$$

1. Estrategia Voraz (Greedy)

La idea intuitiva y óptima consiste en ser lo más "ahorrativo" posible con nuestros recursos (las galletas):

1. **Ordenar** tanto el vector de demandas de los niños (D) como el de tamaños de galletas (G) de menor a mayor.
2. Usar **dos punteros** para recorrer ambos vectores simultáneamente.
3. Intentar satisfacer siempre al niño con la **menor demanda posible** utilizando la **galleta más pequeña disponible** que sea capaz de satisfacerle.
 - Si la galleta actual satisface al niño actual, se la asignamos y pasamos al siguiente niño y a la siguiente galleta.
 - Si la galleta actual es demasiado pequeña para el niño actual, esa galleta no servirá para ningún niño posterior (ya que los siguientes niños tienen demandas aún mayores). Por tanto, descartamos la galleta y probamos con la siguiente.

2. Algoritmo

Pseudocódigo

Entrada: $D[1..n]$ (demandas de los niños), $G[1..m]$ (tamaños de las galletas)

```
// 1. Ordenación previa de menor a mayor
Ordenar_Ascendente(D)
Ordenar_Ascendente(G)
```

```
puntero_niño = 1
puntero_galleta = 1
```

```

niños_satisfechos = 0

// 2. Escaneo voraz con dos punteros
Mientras (puntero_niño <= n e puntero_galleta <= m):
    Si (G[puntero_galleta] >= D[puntero_niño]):
        // La galleta satisface al niño
        niños_satisfechos = niños_satisfechos + 1
        puntero_niño = puntero_niño + 1
        puntero_galleta = puntero_galleta + 1
    Sino:
        // La galleta es demasiado pequeña, la desechamos
        puntero_galleta = puntero_galleta + 1

Retornar niños_satisfechos

```

3. Justificación de Optimalidad (Propiedad Voraz)

Para maximizar el número total de niños contentos, siempre es óptimo asignar la galleta más pequeña que cubra la demanda del niño con menor expectativa:

- Si usáramos una galleta más grande de la necesaria para satisfacer al niño actual, estaríamos desperdiciando innecesariamente un recurso valioso que podría haber sido el único capaz de satisfacer a un niño con una demanda mayor en el futuro.
- Si una galleta g es incapaz de satisfacer al niño con la menor demanda restante, no podrá satisfacer a ningún otro niño restante (ya que todos los demás tienen demandas mayores o iguales). Por lo tanto, desechar g es una decisión segura que no afecta a la cantidad total de niños satisfechos.

4. Análisis de Coste

- **Temporal:**
 - **Ordenación:** Ordenar D cuesta $O(n \log n)$ y ordenar G cuesta $O(m \log m)$.
 - **Búsqueda con dos punteros:** $O(n + m)$, ya que cada elemento de ambos vectores se procesa como máximo una vez.
 - **Coste Total:** $O(n \log n + m \log m)$, donde n es el número de niños y m el de galletas. Este coste está dominado por el tiempo de ordenación.
- **Espacial:** $O(1)$ de memoria extra si la ordenación se realiza *in-place* (sobre los mismos vectores de entrada).